



Openpi script

Field Robotics Laboratory | Minsoo Park

Isaac sim setting

1. Isaac Sim

<https://github.com/MIMI-MINSOO/so101-vla-pipeline>

```
# git-lfs (최초 1회, sudo 필요)
sudo apt-get install -y git-lfs
git lfs install --skip-repo

# IsaacLab (v2.3.2)
git clone --branch v2.3.2 --depth 1 https://github.com/isaac-sim/IsaacLab.git ~/IsaacLab

# LeIsaac fork (IsaacLab/source/ 밑에)
git clone https://github.com/MIMI-MINSOO/leisaac-cupstack.git ~/IsaacLab/source/leisaac
cd ~/IsaacLab/source/leisaac && git lfs pull

# IsaacLab 실행용 venv (python 3.11, uv)
uv venv --python 3.11 ~/IsaacLab/env_isaac
source ~/IsaacLab/env_isaac/bin/activate

pip install -U torch==2.7.0 torchvision==0.22.0 --index-url
https://download.pytorch.org/whl/cu125
pip install torch==2.7.0 torchvision==0.22.0 --index-url https://download.pytorch.org/whl/cu125 --extra-index-url https://pypi.nvidia.com

sudo apt install cmake build-essential # IsaacLab 빌드 의존성
cd ~/IsaacLab && ./isaac.sh --install

pip install -e "source/leisaac[lerobot]"
pip install numpy==1.26.0 # lerobot과 버전 정합
```

```
# openpi (별도 venv, uv)
git clone --recurse-submodules https://github.com/EverNorif/openpi.git
cd openpi && git checkout so101
GIT_LFS_SKIP_SMUDGE=1 uv sync
GIT_LFS_SKIP_SMUDGE=1 uv pip install -e .
```

```
# Isaac-GR00T (별도 venv, python 3.10)
git clone https://github.com/NVIDIA/Isaac-GR00T ~/Isaac-GR00T
cd ~/Isaac-GR00T
git checkout 4af2b622892f7dcb5aae5a3fb70bcb02dc217b96

python3.10 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip setuptools
pip install -e .[base]

# nvcc가 없으면 flash-attn 사전빌드 wheel 사용
pip install "https://github.com/Dao-AI-Lab/flash-attention/releases/download/v2.7.1.post4/flash_attn-2.7.1.post4+cu12torch2.5cxx11abiFALSE-cp310-cp310-linux_x86_64.whl"
```

Isaac sim setting

1. Isaac Sim

<https://github.com/isaac-sim/IsaacLab>

1-1. git clone

```
cd ~  
git clone https://github.com/isaac-sim/IsaacLab.git  
cd IsaacLab  
git checkout v2.3.2
```

1-2. venv

```
uv venv --python 3.11 ~/IsaacLab/env_isaaclab  
source ~/IsaacLab/env_isaaclab/bin/activate
```

1-3. torch → Isaac Sim → IsaacLab

```
pip install --upgrade pip setuptools wheel  
  
pip install -U torch==2.7.0 torchvision==0.22.0 \\  
--index-url https://download.pytorch.org/whl/cu128  
  
pip install "isaacsim[all,extscache]==5.1.0" \\  
--extra-index-url https://pypi.nvidia.com  
  
cd ~/IsaacLab  
./isaacclab.sh --install
```

Isaac sim

2. Leisaac setting

<https://github.com/lightwheelai/leisaac>

2-1. git clone ~/IsaacLab/source/ 아래에 clone

```
cd ~/IsaacLab/source
git clone https://github.com/MIMI-MINS00/leisaac-cupstack.git leisaac
```

2-2. Package install

```
cd ~/IsaacLab/source/leisaac
pip install -e "source/leisaac[lerobot]"
pip install numpy==1.26.0
```

Isaac sim

3. Sim environment setting

[Free Starbucks coffee cup 3D model : papaer cup](#)
[Marker Design 3D : Marker](#)



```
assets/
├─ robots/so101_follower.usd
├─ objects/
│   └─ cup.usd
│   └─ marker/marker.usd
```

```
tasks/
├─ pick_orange/
└─ cup_stack/
```

Copy

```
tasks/
├─ cup_stack/
│   └─ cup_stack_env_cfg.py
│   └─ terminations.py
└─ __init__.py
```

Modify

구분	PickOrange	CupStack
Scene	KITCHEN_WITH_ORANGE_CFG 완성된 kitchen USD 사용	바닥/벽/천장/테이블을 CuboidCfg 등으로 직접 구성
Object	Orange 3개 + Plate	Marker 1개 + Cup 1개
Object loading	parse_usd_and_create_subassets()로 kitchen USD 내부 object 추출	cup.usd, marker.usd를 Rigid ObjectCfg로 직접 spawn
Randomization	오렌지 3개 + plate + front camera pose 랜덤화	marker 위치/yaw만 랜덤화, cup은 고정
Success	세 orange가 plate에 들어갔는지 md p.task_done	marker가 cup 안에 들어갔는지 marker_in_cup

5. Teleoperation (pick_orange)

5-0) keyboard teleoperation

```
#keyboard teleop

python
scripts/environments/teleoperation/teleop_se3_agent.py \
  --task=LeIsaac-S0101-PickOrange-v0 \
  --teleop_device=keyboard \
  --num_envs=1 --device=cuda --enable_cameras
```

B: 에피소드 시작

N: 성공 종료 → 저장

R: 실패 리셋 → 폐기 (파일에 남지 않음)

0. 씬 에셋 준비 (최초 1회)

```
#
https://github.com/LightwheelAI/leisaac/releases/tag/v
0.1.0 에서 "Kitchen with Orange" 다운로드 후
mkdir -p
~/IsaacLab/source/leisaac/assets/scenes/kitchen_with_o
range
# 압축 해제하여 아래 구조가 되도록 배치
#   assets/scenes/kitchen_with_orange/scene.usd
#   assets/scenes/kitchen_with_orange/assets/
#   assets/scenes/kitchen_with_orange/objects/
(Orange001~003, Plate)
```

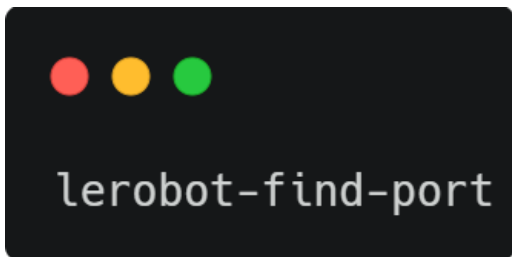
1. Teleop 실행

```
source ~/IsaacLab/env_isaac/bin/activate
cd ~/IsaacLab/source/leisaac
export LEISAAC_ASSETS_ROOT=$(pwd)/assets

python
scripts/environments/teleoperation/teleop_se3_agent.py \
  --task=LeIsaac-S0101-PickOrange-v0 \
  --teleop_device=so101leader \
  --port=/dev/ttyACM0 \
  --num_envs=1 \
  --device=cuda \
  --enable_cameras \
  --record \
  --dataset_file=./datasets/pick_orange/dataset.hdf5
```

옵션	의미
--teleop_device=so101leader	실물 리더암으로 조작
--port	lerobot-find-port로 확인한 leader serial port
--enable_cameras	front/wrist 이미지 기록. VLA 학습에 필요
--record --dataset_file	성공 시연을 HDF5로 저장 (teleop만 할때는 빼)
--resume	(선택) 기존 파일에 이어서 기록

5-1. Port check (leader arm)



5-2 Record (teleoperation)

```

cd ~/IsaacLab/source/leisaac
source ~/IsaacLab/env_isaac/bin/activate
export LEISAAC_ASSETS_ROOT=~/IsaacLab/source/leisaac/assets

python scripts/environments/teleoperation/teleop_se3_agent.py \
    --task=LeIsaac-S0101-CupStack-v0 \
    --teleop_device=so101leader \
    --port=/dev/ttyACM0 \
    --num_envs=1 \
    --device=cuda \
    --enable_cameras \
    --record \
    --dataset_file=./datasets/Marker_97.hdf5

```

Dataset 저장



~/IsaacLab/source/leisaac/datasets/Marker_97.hdf5

B: 에피소드 시작
N: 성공 종료 → 저장
R: 실패 리셋 → 폐기 (파일에 남지 않음)

옵션	의미
--task_name	수집 단계와 동일한 gym id
--repo_id	LeRobot dataset 식별자. 로컬에서는 ~/.cache/huggingface/lerobot/<repo_id>에 저장
--hdf5_files	변환할 HDF5 파일. 여러 세션을 묶을 수도 있음
--task_description	학습에 사용되는 language instruction
--push_to_hub	(선택) 변환 후 Hugging Face Hub 업로드

6-1) HDF5 → LeRobot Dataset (parquet + mp4).

```

cd ~/IsaacLab/source/leisaac
source ~/IsaacLab/env_isaacclab/bin/activate

python scripts/convert/isaacclab2lerobotv3.py \
    --task_name=LeIsaac-S0101-CupStack-v0 \
    --repo_id=mimiminsoo/marker_100 \
    --hdf5_root=./datasets \
    --hdf5_files=Marker_97.hdf5 \
    --task_description="Pick up the marker and place it into the cup,  
then reset the arm to rest state." \
    --enable_cameras
  
```


Isaac sim

7. Openpi install

항목	의미
--recurse-submodules	third_party 등 서브모듈 포함
GIT_LFS_SKIP_SMUDGE=1	openpi가 lerobot을 git 소스로 받을 때 LFS 대용량 파일 다운로드를 건너뛴다
uv sync	openpi 전용 venv를 저장소 안에 생성. env_isaacclab 과 절대 공유하지 않는다

7-1) git clone

```

cd ~
git clone --recurse-submodules https://github.com/EverNorif/openpi.git
cd openpi

GIT_LFS_SKIP_SMUDGE=1 uv sync
GIT_LFS_SKIP_SMUDGE=1 uv pip install -e .

```

src/openpi/training/config.py 에 TrainConfig를 추가한다.

config 이름	데이터	action_horizon	비고
pi0_lora_marker	marker_50	10	초기 실험
pi0_lora_marker_100	marker_100	30	baseline

pi0_lora_marker_100

항목	π0 기본값	marker_100
초기 weight	random init	pi0_base pretrained checkpoint
action horizon	50	30
PaliGemma	non-LoRA	LoRA rank 16 / alpha 16
Action Expert	non-LoRA	LoRA rank 32 / alpha 32
batch size	32	8 (RTX 3090 24GB 기준)
num workers	2	4
train steps	30,000	30,000 (기본값 유지)
optimizer / LR	AdamW / cosine peak 2.5e-5	기본값 유지
dataset	fake/default	mimiminsoo/marker_100

```
TrainConfig(
    name="pi0_lora_marker_100",
    model=pi0_config.Pi0Config(
        action_horizon=30,
        paligemma_variant="gemma_2b_lora",
        action_expert_variant="gemma_300m_lora",
    ),
    data=LeRobotS0101DataConfig(
        repo_id="mimiminsoo/marker_100",

    base_config=DataConfig(prompt_from_task=True),
    ),
    batch_size=8,
    num_workers=4,

    weight_loader=weight_loaders.CheckpointWeightLoader(
        gs://openpi-assets/checkpoints/pi0_base/params"),
    num_train_steps=30_000,
    freeze_filter=pi0_config.Pi0Config(
        action_horizon=30,
        paligemma_variant="gemma_2b_lora",
        action_expert_variant="gemma_300m_lora",
    ).get_freeze_filter(),
    ema_decay=None,
```

9-1) norm_state 계산 + fine-tuning



```
cd ~/openpi
export XLA_PYTHON_CLIENT_MEM_FRACTION=0.9
export HF_HUB_OFFLINE=1

uv run scripts/compute_norm_stats.py --config-name pi0_lora_marker_100
uv run scripts/train.py pi0_lora_marker_100 --exp-name=marker100_h30
```

항목	의미
XLA_PYTHON_CLIENT_MEM_FRACTION=0.9	JAX가 GPU 메모리를 90%까지 선점. 기본 0.75로는 batch size가 부족
HF_HUB_OFFLINE=1	데이터셋이 로컬 캐시에만 있을 때 HF Hub 조회를 건너 뛴다
compute_norm_stats.py	SO-101 dataset의 state/action 정규화 통계 계산. 데이터셋 또는 관련 전처리 config가 바뀌면 다시 계산
--exp-name	체크포인트 디렉토리 이름
--resume	(선택) 마지막 체크포인트에서 이어 학습

옵션	의미
--policy.config	8단계에서 등록한 config 이름
--policy.dir	체크포인트 스텝 디렉토리까지 지정
MEM_FRACTION=0.35	Isaac Sim이 PhysX 씬을 올릴 메모리를 남겨준다. 실로봇만 쓸 때는 높여도 된다

10-1) 정책 서버 open (terminal A)

```

cd ~/openpi
export XLA_PYTHON_CLIENT_MEM_FRACTION=0.35

uv run scripts/serve_policy.py \
    --port 8000 \
    --default-prompt "Pick up the marker and place it into the cup, then reset the
arm to rest state." \
    policy:checkpoint \
    --policy.config=pi0_lora_marker_100 \
    --policy.dir=checkpoints/pi0_lora_marker_100/marker100_h30/29999

```

```

openpi venv                                env_isaacclab venv
serve_policy.py — websocket :8000 —▶ policy_inference.py (11:sim)
                                L▶ real_robot_inference.py (12:real)

```

인자	기본값	이 실험에서 사용	이유
--policy_type	gr00tn1.5	openpi	backend 선택
--policy_port	5555	8000	openpi websocket server port
--policy_action_horizon	16	30	서버가 예측한 action chunk 중 실제 실행할 step 수

11-1) terminal B

```

cd ~/IsaacLab/source/leisaac
source ~/IsaacLab/env_isaac/bin/activate
export LEISAAC_ASSETS_ROOT=$(pwd)

python scripts/evaluation/policy_inference.py \
  --task=LeIsaac-S0101-CupStack-v0 \
  --eval_rounds=20 \
  --seed=42 \
  --policy_type=openpi \
  --policy_host=localhost \
  --policy_port=8000 \
  --policy_action_horizon=30 \
  --policy_language_instruction="Pick up the
marker and place it into the cup, then reset the arm
to rest state." \
  --device=cuda \
  --enable_cameras

```

항목	시물	실기	이유
action horizon	30	10	더 자주 재추론해 closed-loop에 가깝게. 튜닝 대상이며 시물과 맞출 필요 없음
안전장치	없음	--max_relative_target=15.0	정책이 튜는 액션을 내도 한 스텝 이동 폭을 제한

12-1) terminal B

```

cd ~/IsaacLab/source/leisaac
source ~/IsaacLab/env_isaacclab/bin/activate

python scripts/evaluation/real_robot_inference.py \
    --robot_port=/dev/ttyACM0 \
    --front_camera_index=/dev/v4l/by-path/pci-
0000:00:14.0-usb-0:10.2:1.0-video-index0 \
    --wrist_camera_index=/dev/v4l/by-path/pci-
0000:00:14.0-usb-0:10.2:1.0-video-index1 \
    --policy_host=localhost \
    --policy_port=8000 \
    --language_instruction="Pick up the marker and
place it into the cup, then reset the arm to rest
state." \
    --action_horizon=10 \
    --num_episodes=5

```

Isaac sim 최종 실행 순서

```
1. IsaacLab clone (v2.3.2) → ~/IsaacLab
   ↓
2. uv venv → ~/IsaacLab/env_isaaclab
   ↓
3. torch 2.7.0+cu128 → isaacsim 5.1.0 → ./isaacsim.sh --install
   ↓
4. leisaac clone → ~/IsaacLab/source/leisaac (submodule 사용 안 함)
   ↓
5. pip install -e "source/leisaac[1erobot]" → numpy 1.26.0 고정
   ↓
6. asset 확인 (so101_follower / cup / marker) + LEISAAC_ASSETS_ROOT=.../leisaac/assets
   ↓
7. teleop_se3_agent.py → Marker_97.hdf5 (97 episodes / 31776 frames)
   ↓
8. isaacsim21erobotv3.py → mimiminsoo/marker_100 (97ep / 31291f / 30fps)
   ↓
9. openpi clone + uv sync
   ↓
10. config.py 에 pi0_lora_marker_100 등록 (H=30, VLM LoRA r16, Action Expert LoRA r32, batch 8)
   ↓
11. compute_norm_stats.py
   ↓
12. train.py → checkpoints/.../marker100_h30/29999
   ↓
13. serve_policy.py (:8000)
   ↓
14. policy_inference.py → CupStack simulation evaluation
   ↓
15. real_robot_inference.py → 실물 S0-101 배포
```



Thank you



Appendix

[so-101 조립 설명서 - wikidocs](#)

검색어를 입력하세요.

누구나 쉽게 따라하면서 배우는 Physical AI: LeRobot 라이브러리

- 2.2 So-arm 101 조립 가이드
 - 2.3 USB 포트 고정 세팅하기
 - 2.4 카메라 포트 고정 세팅하기
 - 2.5 칼리브레이션
 - 2.6 텔레오퍼레이션
 - 2.7 모방학습을 위한 데이터셋 구축하기
- 3. 데이터셋 다루기
 - 3.1 LeRobotDataset 알아보기
 - 3.2 데이터셋 재생하기
 - 3.3 데이터셋 시각화
 - 3.4 데이터셋 편집 도구
 - 3.5 데이터셋 불러오기와 활용하기
- 4. 정책 학습하고 추론하기
 - 4.0 Policy 훈련 및 추론 - 공통 옵션
 - 4.1 ACT
 - 4.2 SmoVLA
 - 4.2 SmoVLA

2.2 So-arm 101 조립 가이드

1. 첫 번째 모터를 베이스에 넣고, M2×6mm 나사 4개(위 2개, 아래 2개)로 고정합니다.
2. 모터 홀더를 끼우고, M2×6mm 나사 2개(양쪽 각 1개)로 고정합니다.
3. 모터 혼 2개를 설치하고, 상단 혼은 M3×6mm 나사로 고정합니다.
4. 솔더 부품을 부착하고, M3×6mm 나사 4개(위 4개, 아래 4개)로 고정합니다.
5. 솔더 모터 홀더를 추가합니다.



Joint 2 (솔더)